



北京邮电大学
Beijing University of Posts and Telecommunications

北京邮电大学

计算机学院

Python 链家五大城市租房数据爬取与分析实验报告

姓名: 吴清柳

学号: 2020211597

班级: 2020211323

指导老师: 邵莹侠

课程名称: 算法设计与分析

December 31, 2022

Contents

1	概述	3
2	爬虫设计和数据爬取	3
2.1	行政区划和板块爬虫	3
2.1.1	爬取结果的持久化	3
2.2	小区爬虫	4
2.2.1	已爬标记和翻页	4
2.2.2	爬取结果的持久化	4
2.3	租房房源爬虫	4
2.3.1	已爬标记和翻页	5
2.3.2	爬取结果的持久化	5
2.4	部署在 scrapyd 上进行爬取	5
3	数据分析和处理	5
4	实验总结	6
5	附录	6

1 概述

在本实验中,选取北京,上海,广州,深圳和潍坊五个城市,借助 Scrapy 框架设计爬虫,部署在 Scrapyd 上运行爬取;爬取数据以定义的 item 结构保存,通过 pipeline 传输到使用 SQLite3 建立的数据库进行去重,断点续爬和持久化.使用 pandas 与 matplotlib 进行数据分析和可视化.

由于链家 100 页只有 3000 个房源,远小于房源总数,直接对租房房源进行爬取将遗漏大量房源信息.因此为了尽可能多的爬取房源,制作了三个爬虫.分别爬取板块(商圈, business area),小区(community)和租房房源(rental).在爬取板块的同时,爬虫可以自动收集行政区划,小区指北京邮电大学家属小区这一级,租房房源指具体的出租房源信息.由于一个小区内不会有超过 3000 各租房房源,因此通过这一划分方式,可以尽可能的避免这一限制造成的爬取不充分.

由于实现了上述的较高程度的自动化,爬虫对城市的增加有极强的扩展性.如果需要添加新的城市,只需要添加新城市的租房 url 即可,而不需要手动添加待添加城市的具体行政区划等信息.

爬虫共计爬取商圈 800 个,其中北京 255 个,上海 172 个,广州 227 个,深圳 74 个,潍坊 72 个.小区 41262 个,其中上海 17211 个,北京 10213 个,广州 6575 个,深圳 4098 个,潍坊 3165 个.租房房源共 175570 个,其中北京 31095 个,上海 22998 个,广州 74387 个,深圳 37840 个,潍坊 9250 个.由于商圈,小区和房源的 url 中都有其唯一编码,故在插入 SQLite 的时候借助 unique 关键字对其进行去重.

2 爬虫设计和数据爬取

在本实验中,选取北京,上海,广州,深圳和潍坊五个城市,借助 Scrapy 框架设计爬虫,部署在 Scrapyd 上运行爬取;爬取数据以定义的 item 结构保存,通过 pipeline 传输到使用 SQLite3 建立的数据库进行去重,断点续爬和持久化.

爬虫设置方面,由于部署在 Scrapyd 上进行爬取,因此爬取时间不成为限制.为了避免可能出现的反爬措施,设置下载延迟为 610 秒,每个域名并行请求数为 2,每个 IP 并行请求数为 2,并使用了轮换的 User Agent.

由于链家一次只显示 100 页,只有 3000 个房源,远小于房源总数,直接对租房房源进行爬取将遗漏大量房源信息.因此为了尽可能多的爬取房源,制作了三个爬虫,分别爬取板块(商圈, business area),小区(community)和租房房源(rental).在爬取板块的同时,爬虫可以自动收集行政区划,小区指北京邮电大学家属小区这一级,租房房源指具体的出租房源信息.由于一个小区内不会有超过 3000 各租房房源,因此通过这一划分方式,可以尽可能的避免这一限制造成的爬取不充分.对三个爬虫分别介绍如下.

2.1 行政区划和板块爬虫

这一爬虫是首先运行的爬虫,定义在“business_area_spider.py”中的 `class BusinessAreaSpider` 中.

其部署到 scrapyd 或者本地运行后,将以 start_requests 中 urls 中的 url 为起点,对每个城市的行政区,以及板块进行爬取.

2.1.1 爬取结果的持久化

爬取到的板块将保存在定义在“items.py”中的 `class BusinessAreaItem` 中,并传输给定义在“pipeline.py”中的 `class DownBusinessAreaUrlPipeline`,由该 pipeline 判断此 BusinessAreaItem 所在的城市,行政区,以及此板块是否分别存在与相应数据库表中.如果不存在,则插入.

BusinessAreaItem 定义如下.

```
class BusinessAreaItem(scrapy.Item):  
    # Business area
```

```
business_area_url = scrapy.Field()
business_area_name = scrapy.Field()
business_area_region = scrapy.Field()
business_area_city = scrapy.Field()
```

分别存储了板块的 url, 名称, 行政区和所在城市. 而其保存到的 SQL 中的板块表还有一个 accessbit 字段, 用于判断该板块是否已经被下面介绍的小区爬虫爬取过. 借助这一字段, 可以方便的实现断点续爬: 不会对已经爬取过的小区重复爬取, 而是对因为意外中断而没有来得及爬取的板块中小区进行爬取.

2.2 小区爬虫

这一爬虫在行政区划和板块爬虫爬取完毕后运行, 定义在“community_spider.py”中的 `class ComemunitySpider` 在本地或者 scrapyd 上启动后, 将首先访问 SQLite 数据库, 获取没有访问的板块 url 和对应城市 url, 并对其开始并行爬取. 由于小区数量很多, 一页不能完全展示, 所以需要处理翻页问题; 此外, 为了避免对一个区块的重复爬取, 需要设置已爬标记.

2.2.1 已爬标记和翻页

每爬取完一页小区后, 需要借助当前页码和总页数进行判断. 当前页和总页数均存储在 `response.xpath("//div[@class='house-list-page-box']/@pagedata")` 中. 如果已经是最后一页小区, 则设置该小区所在的 business_area 的 accessbit 为 1, 表示这一区块已经被爬取过. 如果不是最后一页小区, 则对 response.url 进行处理, 提取出当前板块的主 url, 并链接 `f"pg{next_page}/"` 字段, 作为下一页 url 进行爬取.

2.2.2 爬取结果的持久化

爬取结果保存在 `class CommunityItem` 中, 传输给 `class DownCommunityInfoPipeline`, 由其根据小区链接 `item["community_url"]` 是否已在数据库中来判断该结果是否重复. 如果不重复, 则插入数据库. `CommunityItem` 定义如下.

```
class CommunityItem(scrapy.Item):
    # Community
    community_url = scrapy.Field()
    community_name = scrapy.Field()
    community_region = scrapy.Field()
    community_city = scrapy.Field()
    community_business_area = scrapy.Field()
    community_rent_url = scrapy.Field()
```

分别存储小区链接, 小区名, 小区所在行政区, 小区所在城市, 小区所在板块和小区租房链接. 其中如果小区内没有房屋出租, 则租房链接为 'None', 避免数据库中出现 null 值.

2.3 租房房源爬虫

小区信息爬取完毕后, 在小区信息的基础上进行租房房源信息的爬取. 租房房源爬虫为“rent_spider.py”中的 `class RentSpider(scrapy.Spider)`, 首先从数据库中读取租房链接 `community_rent_url` 非 'None', 且访问位 `community_accessbit` 为 0 的小区列表, 再对这些小区进行租房房源的并行爬取. 由于单小区内

房源也会有一页不能完全展示的问题, 因此也需要处理翻页问题. 链家的租房房源不同页的 url 设计比较奇怪, 给翻页问题的处理带来了一些困扰.

2.3.1 已爬标记和翻页

与爬取板块内小区不同的是, 小区内会出现没有房源正在出租的情况. 通过仔细观察, 注意到字段 `response.xpath("//span[@class='q']/text())` 记录了当前小区内房源数量. 如果当前小区内租房房源数量为 0, 则设置当前小区的访问位 `community_accessbit` 为 2, 跳过当前小区.

否则, 每爬取完小区内的一页租房房源后, 需要借助当前页码和总页数进行判断. 当前页和总页数均存储在 `response.xpath("//div[@class='content__pg']")` 中. 如果已经是最后一页小区, 则设置该页所在小区的 `community_accessbit` 为 1, 结束对该小区的访问.

2.3.2 爬取结果的持久化

爬取结果保存在 `class RentalItem` 中, 传输给 `class DownRentalInfoPipeline`, 由其根据房源 url 判断是否已经在 SQLite 数据库中的 `rental` 表中. 如果不存在, 则插入. `RentalItem` 定义如下:

```
class RentalItem(scrapy.Item):
    # Rental info
    rental_name = scrapy.Field()
    rental_url = scrapy.Field()
    rental_city=scrapy.Field()
    rental_region = scrapy.Field()
    rental_business_area = scrapy.Field()
    rental_community_url=scrapy.Field()
    rental_community = scrapy.Field()
    rental_area = scrapy.Field()
    rental_lighting = scrapy.Field()
    rental_rooms = scrapy.Field()
    rental_liverooms = scrapy.Field()
    rental_bathrooms = scrapy.Field()
    rental_price = scrapy.Field()
    rental_timestamp = scrapy.Field()
```

2.4 部署在 scrapyd 上进行爬取

完成上述过程后, 为了让爬虫能够稳定运行, 在服务器上安装 scrapyd, 部署在端口 6800 并转发端口. 在本地使用 scrapyd-client 将爬虫部署到服务器的 scrapyd, 并依次运行 `BusinessAreaSpider`, `CommunitySpider` 和 `RentSpider` 进行爬取.

3 数据分析和处理

链家租房数据分析

数据导入和预处理

数据导入

配置数据分析工具

导入numpy, pandas和matplotlib, 配置字体, 行内显示matplotlib图.

```
import numpy as n
import seaborn as sns
import pandas as pd
from pandas import Series, DataFrame
import matplotlib.pyplot as plt
sns.set_style({'font.sans-serif': ['simhei', 'Arial']})

%matplotlib inline

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

# Config global font
plt.rcParams['font.sans-serif']='Songti SC'
plt.rcParams['axes.unicode_minus'] =False
```

导入链家租房数据.

```
lianjia_data = pd.read_csv("./database/rental.csv")
display(lianjia_data.head(), lianjia_data.shape)
```

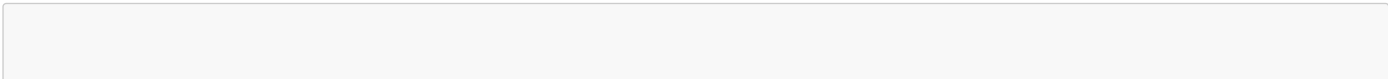
```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	rental_id	rental_name	rental_url	rental_city	rental_region	rental_business_area	rental_communi
0	1	整租·海语山林 2室2厅 东南	https://sz.lianjia.com/zufang/SZ17043626101466...	深圳	大鹏新区	大鹏半岛	https://sz.lianjia.c
1	2	整租·海语山林 2室2厅 南	https://sz.lianjia.com/zufang/SZ17123504436207...	深圳	大鹏新区	大鹏半岛	https://sz.lianjia.c
2	3	整租·海语山林 3室2厅 跃层 南	https://sz.lianjia.com/zufang/SZ16540747918601...	深圳	大鹏新区	大鹏半岛	https://sz.lianjia.c
3	4	整租·海语山林 3室2厅 南	https://sz.lianjia.com/zufang/SZ17028886905623...	深圳	大鹏新区	大鹏半岛	https://sz.lianjia.c
4	5	整租·海语山林 3室2厅 东南	https://sz.lianjia.com/zufang/SZ15809179083834...	深圳	大鹏新区	大鹏半岛	https://sz.lianjia.c

```
(170039, 16)
```

查看数据概况



```
display(lianjia_data.info(), lianjia_data.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 170039 entries, 0 to 170038
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   rental_id              170039 non-null  int64
1   rental_name            170039 non-null  object
2   rental_url             170039 non-null  object
3   rental_city            170039 non-null  object
4   rental_region          170039 non-null  object
5   rental_business_area   170039 non-null  object
6   rental_community_url    170039 non-null  object
7   rental_community       170039 non-null  object
8   rental_area            170039 non-null  float64
9   rental_lighting        170039 non-null  object
10  rental_rooms           170039 non-null  int64
11  rental_liverooms       170039 non-null  int64
12  rental_bathrooms       170039 non-null  int64
13  rental_price           170039 non-null  int64
14  rental_timestamp       170039 non-null  object
15  rental_accessbit       170039 non-null  int64
dtypes: float64(1), int64(6), object(9)
memory usage: 20.8+ MB

None
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	rental_id	rental_area	rental_rooms	rental_liverooms	rental_bathrooms	rental_price	rental_accessbit
count	170039.000000	170039.000000	170039.000000	170039.000000	170039.000000	170039.000000	170039.0
mean	85020.000000	83.868990	2.366404	1.300478	1.303054	5856.955634	0.0
std	49086.175549	541.967185	1.084734	0.640657	0.611181	7730.758572	0.0
min	1.000000	3.000000	0.000000	0.000000	0.000000	150.000000	0.0
25%	42510.500000	50.000000	2.000000	1.000000	1.000000	2500.000000	0.0
50%	85020.000000	76.000000	2.000000	1.000000	1.000000	4200.000000	0.0
75%	127529.500000	100.000000	3.000000	2.000000	2.000000	6500.000000	0.0
max	170039.000000	201306.000000	20.000000	9.000000	9.000000	450000.000000	0.0

可以看到, 得益于SQLite数据库的not null constraint, 没有null的数据存在。

但是租房面积最小为3平米, 最大为201306平米. 由于房子的宜居面积有推荐值, 所以认为 租房面积为正态分布, 故超出3倍标准差范围的过大面积房屋初步认为是异常值。

字段处理

观察发现, 房屋主键rental_id, 房屋访问位rental_accessbit, 小区url等字段都是用来服务爬虫的, 对本次数据分析和数据可视化没有作用, 所以去掉。

添加单位面积租金(unit_price)字段, 去掉字段中的lianjia_前缀. 此前缀是用来在数据库中区分不同表的, 在数据分析中没有用。

```
df = lianjia_data.copy()

df["unit_price"] = (
    lianjia_data["rental_price"] / lianjia_data["rental_area"]
).round(2)
```

```
df.drop(["rental_id"], axis=1, inplace=True)

df = df.rename(
    columns={
        "rental_name": "name",
        "rental_community": "community",
        "rental_business_area": "business_area",
        "rental_region": "region",
        "rental_city": "city",
        "rental_area": "area",
        "rental_price": "price",
        "rental_lighting": "lighting",
        "rental_rooms": "rooms",
        "rental_liverooms": "liverooms",
        "rental_bathrooms": "bathrooms",
    }
)

# Refactor columns
columns = [
    "name",
    "community",
    "business_area",
    "region",
    "city",
    "area",
    "price",
    "unit_price",
    "lighting",
    "rooms",
    "liverooms",
    "bathrooms",
]

df = pd.DataFrame(df, columns=columns)

df = df[~df['region'].isin(['海珠区', '南山', '宝安'])]

display(df.describe(), df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 170036 entries, 0 to 170038
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   name             170036 non-null object
1   community        170036 non-null object
2   business_area    170036 non-null object
3   region           170036 non-null object
4   city             170036 non-null object
5   area             170036 non-null float64
6   price            170036 non-null int64
7   unit_price       170036 non-null float64
8   lighting         170036 non-null object
9   rooms            170036 non-null int64
10  liverooms        170036 non-null int64
11  bathrooms        170036 non-null int64
dtypes: float64(2), int64(4), object(6)
memory usage: 16.9+ MB
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	area	price	unit_price	rooms	liverooms	bathrooms
count	170036.000000	170036.000000	170036.000000	170036.000000	170036.000000	170036.000000

	area	price	unit_price	rooms	liverooms	bathrooms
mean	83.870193	5857.031452	78.047265	2.366358	1.300489	1.303042
std	541.971890	7730.805523	53.368977	1.084687	0.640645	0.611179
min	3.000000	150.000000	0.020000	0.000000	0.000000	0.000000
25%	50.000000	2500.000000	40.280000	2.000000	1.000000	1.000000
50%	76.000000	4200.000000	66.150000	2.000000	1.000000	1.000000
75%	100.000000	6500.000000	103.230000	3.000000	2.000000	2.000000
max	201306.000000	450000.000000	997.010000	20.000000	9.000000	9.000000

None

查找异常数据

借助skewness值来判断异常数据. 在一个城市中, 单位面积租金和租金价格加入服从正态分 布, 则理想的skewness值应当在-1和1之间. 超出的值则是极端值.

```
cities = list(df["city"].drop_duplicates())
for city in cities:
    curr_df = df[df["city"] == city]
    print(city, "price.skew:", curr_df["price"].skew())
    print(city, "unit_price.skew:", curr_df["unit_price"].skew())
```

深圳 price.skew: 12.885845104991153
深圳 unit_price.skew: 1.8363484385731965
潍坊 price.skew: 23.4630552187306
潍坊 unit_price.skew: 7.291901891053477
上海 price.skew: 8.011806137717477
上海 unit_price.skew: 1.7474531801646254
北京 price.skew: 11.092843846861996
北京 unit_price.skew: 1.8494180991821851
广州 price.skew: 14.811631702838538
广州 unit_price.skew: 3.2848932454809536

可以看到, 租金价格明显不符合正态分布, 而单位价格相对来说更贴近正态分布. 下面将对全部数据和去除异常值后的数据分别绘图进行展示和分析.

5城市房租情况比较和展示

比较 5 个城市的总体房租情况, 包含租金的均价、最高价、最低价、中位数 等信息, 单位 面积租金(元/平米)的均价、最高价、最低价、中位数等信息. 采用合适的图或表形式进行 展示.

租房价格比较

首先展示五城市的租房价格均价, 最高价, 最低价, 和中位数.

```
price_df = pd.DataFrame(df, columns=["city", "price"])
mean = price_df["price"].mean()
std = price_df["price"].std()
upper_border = mean + 3 * std

for city in cities:
    print(f"{city}:")
    print(f"\t全部数据统计信息:")
    display(
        price_df.loc[price_df["city"] == city]
        .agg({"price": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"price": "价格"},axis=1)
    )
    print(f"\t三倍标准差内数据统计信息:")
    display(
        price_df.loc[
            (price_df["city"] == city) & (price_df["price"] < upper_border)
        ]
        .agg({"price": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"price": "价格"},axis=1),
```

)

深圳：
全部数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	价格
平均值	7213.779834
最大值	450000.000000
最小值	900.000000
中位数	5300.000000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	价格
平均值	6249.442526
最大值	29000.000000
最小值	900.000000
中位数	5200.000000

潍坊：
全部数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	价格
平均值	1486.871712
最大值	69999.000000
最小值	200.000000
中位数	1300.000000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	价格
平均值	1471.445203
最大值	29000.000000
最小值	200.000000
中位数	1300.000000

上海:
全部数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	价格
平均值	7925.590222
最大值	250000.000000
最小值	1000.000000
中位数	5800.000000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	价格
平均值	7149.887654
最大值	29000.000000
最小值	1000.000000
中位数	5800.000000

北京:
全部数据统计信息:

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	价格
平均值	9060.25783
最大值	450000.00000
最小值	500.00000
中位数	6600.00000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	价格
平均值	7845.312112
最大值	29000.000000
最小值	500.000000
中位数	6500.000000

广州:
全部数据统计信息:

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	价格
平均值	3817.837815
最大值	240000.000000
最小值	150.000000
中位数	3000.000000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
```

```
        text-align: right;
    }
```

	价格
平均值	3610.282265
最大值	29000.000000
最小值	150.000000
中位数	3000.000000

分别绘制了全部数据和去除三倍标准差外数据的图进行比较.

```
mean = price_df["price"].mean()
std = price_df["price"].std()
upper_border = mean + 3 * std

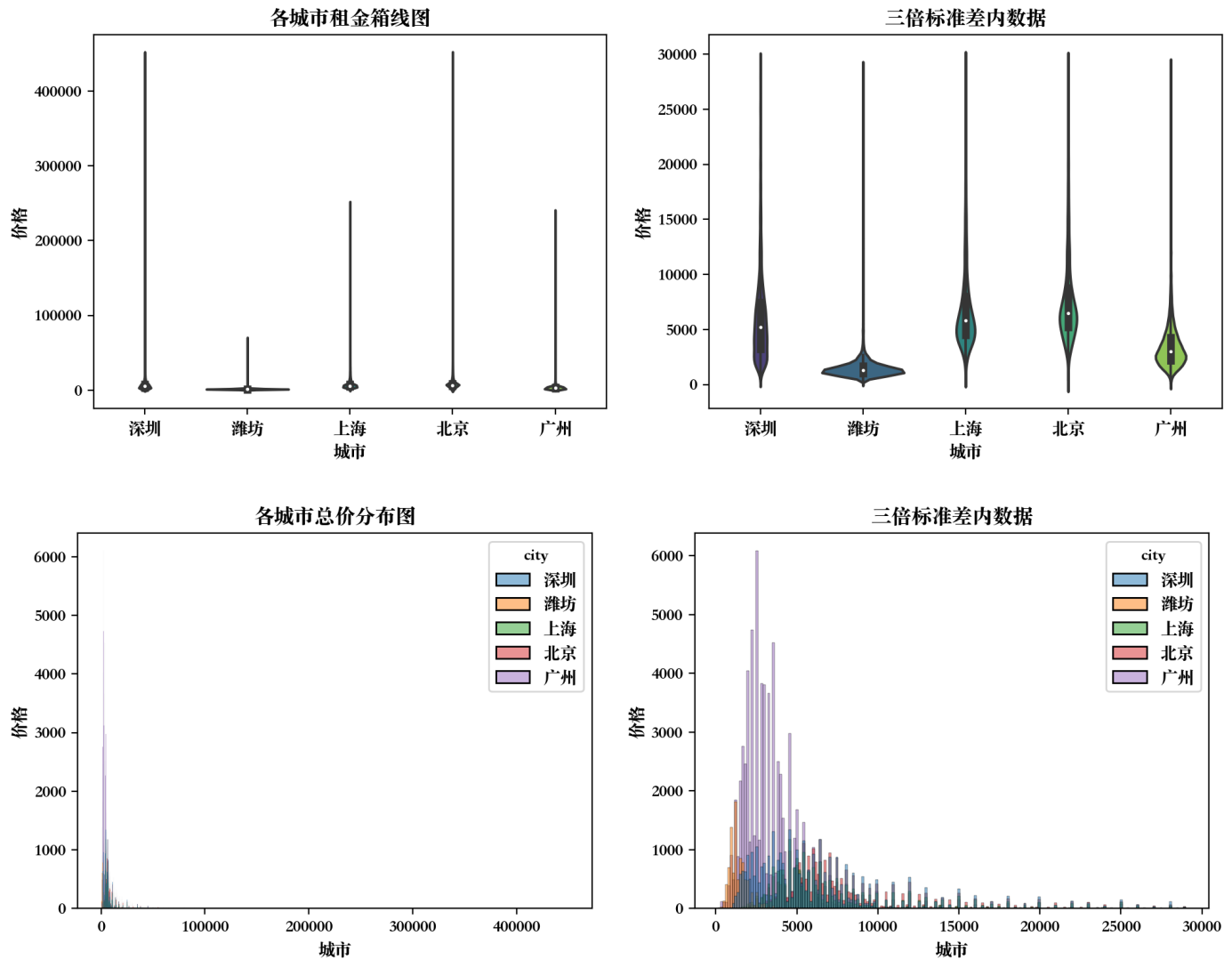
print(price_df["price"].skew())

cities = list(price_df["city"].drop_duplicates())

fig, [ax1, ax2] = plt.subplots(1, 2, figsize=(12, 4), dpi=200)
g = sns.violinplot(
    x="city", y="price", data=price_df, palette="viridis", ax=ax1
)
g.set_title("各城市租金箱线图")
g.set_xlabel("城市")
g.set_ylabel("价格")
g = sns.violinplot(
    x="city",
    y="price",
    data=price_df[price_df["price"] < upper_border],
    palette="viridis",
    ax=ax2,
)
g.set_title("三倍标准差内数据")
g.set_xlabel("城市")
g.set_ylabel("价格")
plt.show()

fig, [ax1, ax2] = plt.subplots(1, 2, figsize=(12, 4), dpi=200)
g = sns.histplot(x="price", hue="city", data=price_df, ax=ax1)
g.set_title(
    "各城市总价分布图"
)
g.set_xlabel("城市")
g.set_ylabel("价格")
g = sns.histplot(
    x="price",
    hue="city",
    data=price_df[price_df["price"] < upper_border],
    ax=ax2,
)
g.set_title("三倍标准差内数据")
g.set_xlabel("城市")
g.set_ylabel("价格")
plt.show()
```

12.6257039780833



可以看到, 在不去除异常值的情况下, 小提琴图中有大量超出whisker的1.5倍IQR的数值, 平均总价分布histogram中也显示, 租房价格极高的房源极少, 造成明显的长尾分布。

具体到城市的区别, 北京, 上海和深圳的分布比较类似, 分布上更偏向价格更高的部分; 而广州和潍坊的租金相对更加低廉。

租房单价比较

与上面租房价格的比较流程相同. 首先展示五大城市的租房价格统计数据。

```
unit_price_df = pd.DataFrame(df, columns=["city", "unit_price"])
mean = unit_price_df["unit_price"].mean()
std = unit_price_df["unit_price"].std()
upper_border = mean + 3 * std

for city in cities:
    print(f"{city}:")
    print(f"\t全部数据统计信息:")
    display(
        unit_price_df.loc[unit_price_df["city"] == city]
        .agg({"unit_price": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"unit_price": "均价"}, axis=1),
    )

    print(f"\t三倍标准差内数据统计信息:")
    display(
        unit_price_df.loc[
            (unit_price_df["city"] == city)
            & (unit_price_df["unit_price"] < upper_border)
        ]
        .agg({"unit_price": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"unit_price": "均价"}, axis=1),
    )
```

深圳：
全部数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	均价
平均值	113.498366
最大值	933.330000
最小值	1.000000
中位数	99.430000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	均价
平均值	103.867782
最大值	238.100000
最小值	1.000000
中位数	96.150000

潍坊：
全部数据统计信息：

```
.dataframe tbody tr th {  
    vertical-align: top;  
}  
  
.dataframe thead th {  
    text-align: right;  
}
```

	均价
平均值	15.308537
最大值	283.330000
最小值	0.850000
中位数	13.950000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	均价
平均值	15.279518
最大值	85.000000
最小值	0.850000
中位数	13.950000

上海：
全部数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	均价
平均值	101.186555
最大值	670.730000
最小值	8.300000
中位数	92.935000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	均价
平均值	98.275846
最大值	238.120000
最小值	8.300000
中位数	92.200000

北京：
全部数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}
```



```
.dataframe thead th {
  text-align: right;
}
```

	均价
平均值	100.735278
最大值	997.010000
最小值	0.020000
中位数	96.845000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	均价
平均值	99.550303
最大值	238.100000
最小值	0.020000
中位数	96.550000

广州：
全部数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	均价
平均值	52.231509
最大值	909.090000
最小值	0.020000
中位数	45.750000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	均价
--	----

	均价
平均值	51.554925
最大值	238.100000
最小值	0.020000
中位数	45.710000

分别绘制全部数据和去除三倍标准差外数据的图进行比较.

```
print(unit_price_df['unit_price'].skew())
print(unit_price_df[unit_price_df['unit_price'] < upper_border].skew())
```

```
1.8721898687998964
unit_price      0.928964
dtype: float64
```

通过上面的skewness可以看到, 即便在不去除三倍标准差外数据的情况下, 单位面积租金也比租金总价更贴近正态分布.

```
mean = unit_price_df["unit_price"].mean()
std = unit_price_df["unit_price"].std()
upper_border = mean + 3 * std

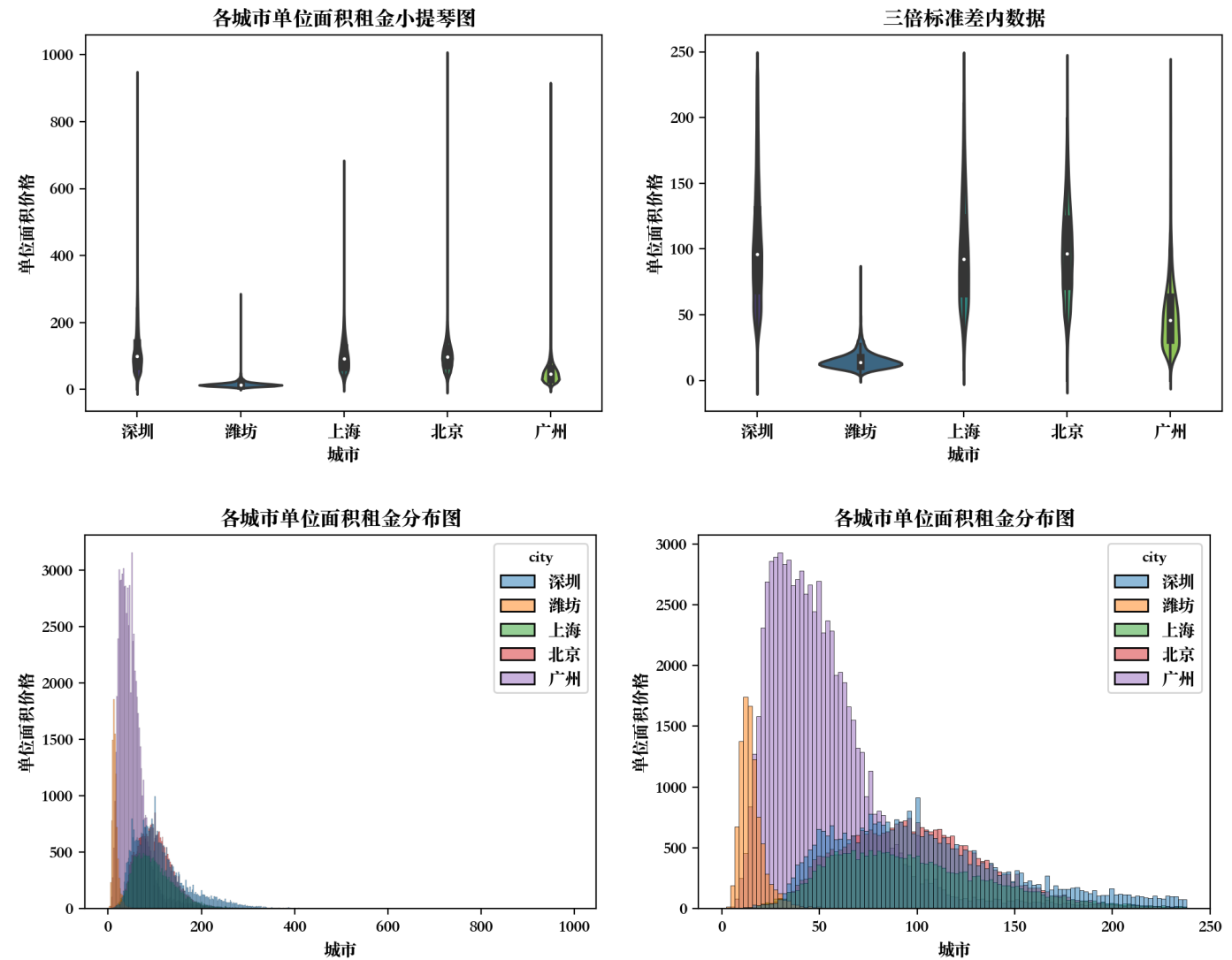
fig, [ax1, ax2] = plt.subplots(1, 2, figsize=(12, 4), dpi=200)

g = sns.violinplot(
    x="city", y="unit_price", data=unit_price_df, palette="viridis", ax=ax1
)
g.set_title("各城市单位面积租金小提琴图")
g.set_xlabel("城市")
g.set_ylabel("单位面积价格")

g = sns.violinplot(
    x="city",
    y="unit_price",
    data=unit_price_df[(unit_price_df["unit_price"] < upper_border)],
    palette="viridis",
    ax=ax2,
)
g.set_title("三倍标准差内数据")
g.set_xlabel("城市")
g.set_ylabel("单位面积价格")
plt.show()

fig, [ax1, ax2] = plt.subplots(1, 2, figsize=(12, 4), dpi=200)
g = sns.histplot(x="unit_price", hue="city", data=unit_price_df, ax=ax1)
g.set_title("各城市单位面积租金分布图")
g.set_xlabel("城市")
g.set_ylabel("单位面积价格")

g = sns.histplot(
    x="unit_price",
    hue="city",
    data=unit_price_df[unit_price_df["unit_price"] < upper_border],
    ax=ax2,
)
g.set_title("各城市单位面积租金分布图")
g.set_xlabel("城市")
g.set_ylabel("单位面积价格")
plt.show()
```



可以看到, 北京, 上海和深圳的单位面积租金相对更高, 中位数, 均值, 四分之一数, 四分 之三数等均高于广州和潍坊. 北京和上海的分布非常相似. 广州和潍坊的单位面积租金更低, 分布更集中.

分析和比较各居室房型

比较5个城市的一居, 二居和三局的租金均价, 最高价, 最低价和中位数等信息.

居室信息在Scrapy保存对SQLite数据库的时候就已经分离完毕, 在liverooms字段中, 因此只需直接提取即可进行比较.

```
# Get liverooms DataFrame
rooms_df = pd.DataFrame(df, columns=['city', 'rooms'])
mean = rooms_df['rooms'].mean()
std=rooms_df['rooms'].mean()
upper_border=mean+3*std

for city in cities:
    print(f"{city}:")
    print(f"\t全部数据统计信息:")
    display(
        rooms_df.loc[rooms_df["city"] == city]
        .agg({"rooms": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"rooms": "居室数量"},axis=1)
    )
    print(f"\t三倍标准差内数据统计信息:")
    display(
        rooms_df.loc[
            (rooms_df["city"] == city) & (rooms_df["rooms"] < upper_border)
        ]
        .agg({"rooms": ["mean", "max", "min", "median"]})
        .rename({"mean": "平均值", "max": "最大值", "min": "最小值", "median": "中位数"})
        .rename({"rooms": "居室数量"},axis=1),
    )
```

深圳：
全部数据统计信息：

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.683549
最大值	15.000000
最小值	0.000000
中位数	3.000000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.676047
最大值	9.000000
最小值	0.000000
中位数	3.000000

潍坊：
全部数据统计信息：

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.452095
最大值	9.000000
最小值	0.000000
中位数	3.000000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	居室数量
平均值	2.452095
最大值	9.000000
最小值	0.000000
中位数	3.000000

上海：
全部数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	居室数量
平均值	2.034813
最大值	10.000000
最小值	1.000000
中位数	2.000000

三倍标准差内数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	居室数量
平均值	2.034439
最大值	9.000000
最小值	1.000000
中位数	2.000000

北京：
全部数据统计信息：

```
.dataframe tbody tr th {
    vertical-align: top;
}
```

```
.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.116116
最大值	13.000000
最小值	0.000000
中位数	2.000000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.114893
最大值	9.000000
最小值	0.000000
中位数	2.000000

广州：
全部数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
平均值	2.396116
最大值	20.000000
最小值	0.000000
中位数	2.000000

三倍标准差内数据统计信息:

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

	居室数量
--	------

	居室数量
平均值	2.395654
最大值	9.000000
最小值	0.000000
中位数	2.000000

平均值方面, 可以看到相比于北京上海, 另外三各城市的居室数量平均要更高一些. 这也和 北京上海更高的单位面积价格相关.

最大值方面, 对全部数据, 广州的居室最大数量最大, 为20个, 其次是深圳; 对三倍标准差 内数据, 则均为9个.

最小值方面, 都是0个.

各城市居室数量可视化分析

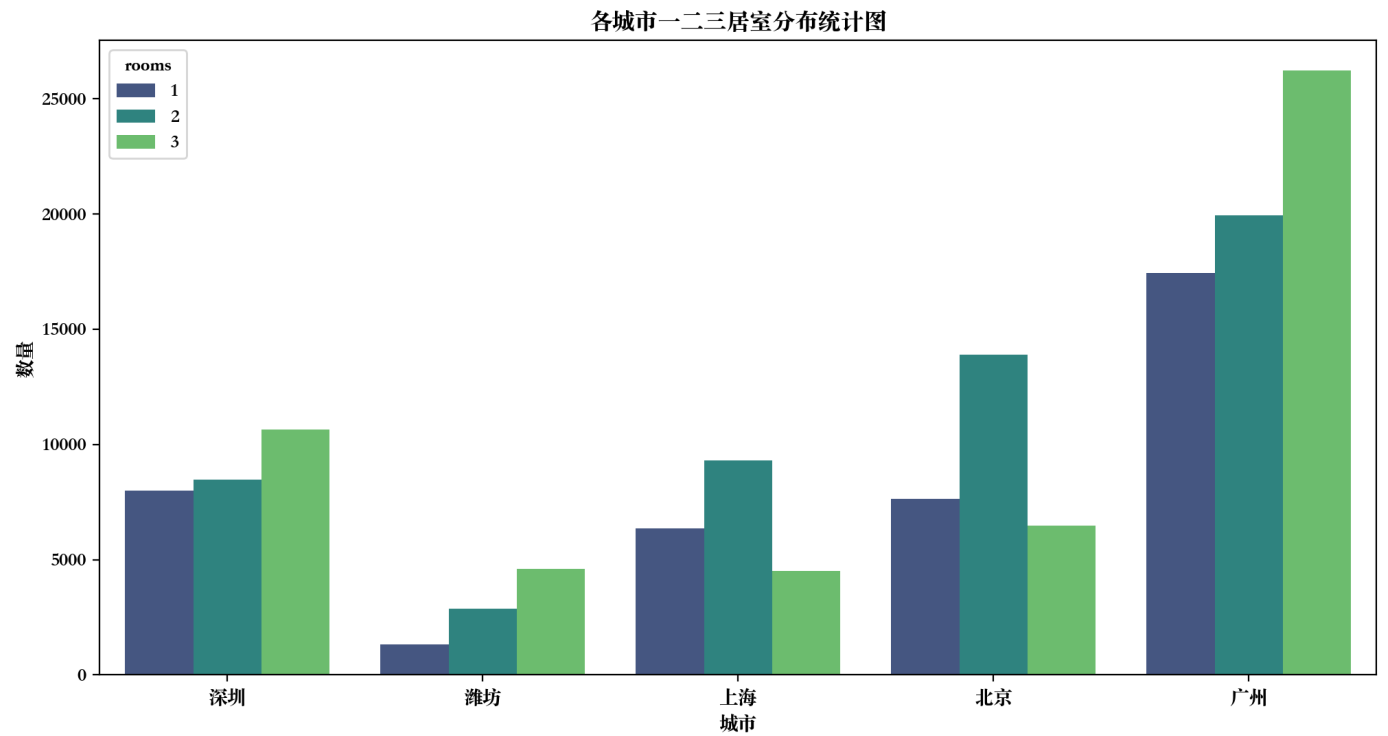
首先筛选出一居, 二居和三局的房型, 再绘制统计图进行比较.

```
filtered_rooms_df = rooms_df[rooms_df['rooms'].isin([1,2,3])]
display(filtered_rooms_df['rooms'].value_counts(ascending=False))
```

```
2      54499
3      52447
1      40745
Name: rooms, dtype: int64
```

```
mean = filtered_rooms_df["rooms"].mean()
std = filtered_rooms_df["rooms"].std()
upper_border = mean + 3 * std

fig, ax = plt.subplots(figsize=(12, 6), dpi=200)
g = sns.countplot(
    x="city", data=filtered_rooms_df, hue="rooms", palette="viridis", ax=ax
)
g.set_title("各城市一二三居室分布统计图")
g.set_xlabel("城市")
g.set_ylabel("数量")
plt.show()
```

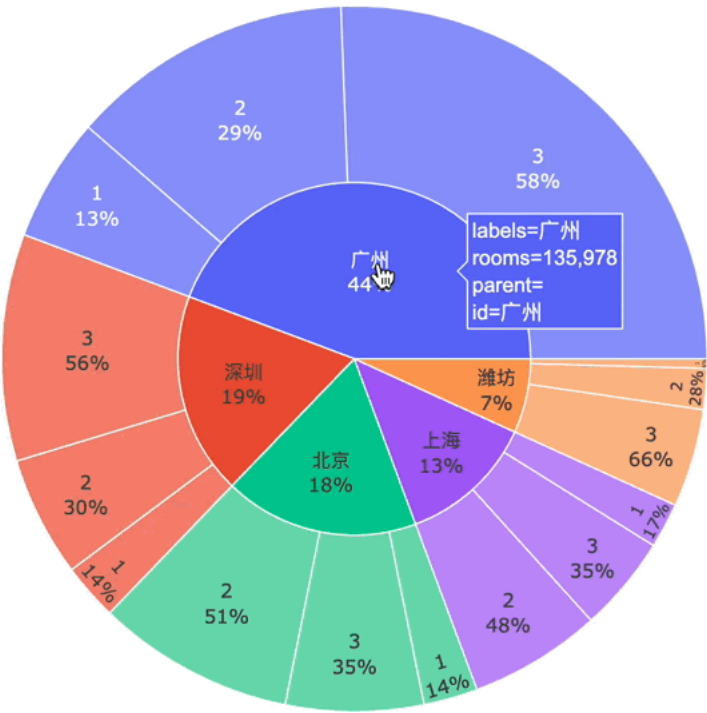


使用饼图来更方便的表示一二三居室在每个城市中的占比. 为了更方便的展示, 生成可交互饼图.

```
import plotly.express as px

figure = px.sunburst(
    filtered_rooms_df, values="rooms", path=["city", 'rooms'], width=600, height=600
)

figure.update_traces(textinfo='label+percent parent')
figure.show()
```



上述代码生成的可交互饼图如下:

板块均价分析

对各个城市的板块均价进行统计分析. 计算和分析各个城市不同板块的均价, 并绘图分析.

首先对各城市行政区租售价格使用箱性图进行刻画. 为更好的突出平均值特征, 不显示箱性图中的outliers, 且忽略三倍标准差之外的数据.

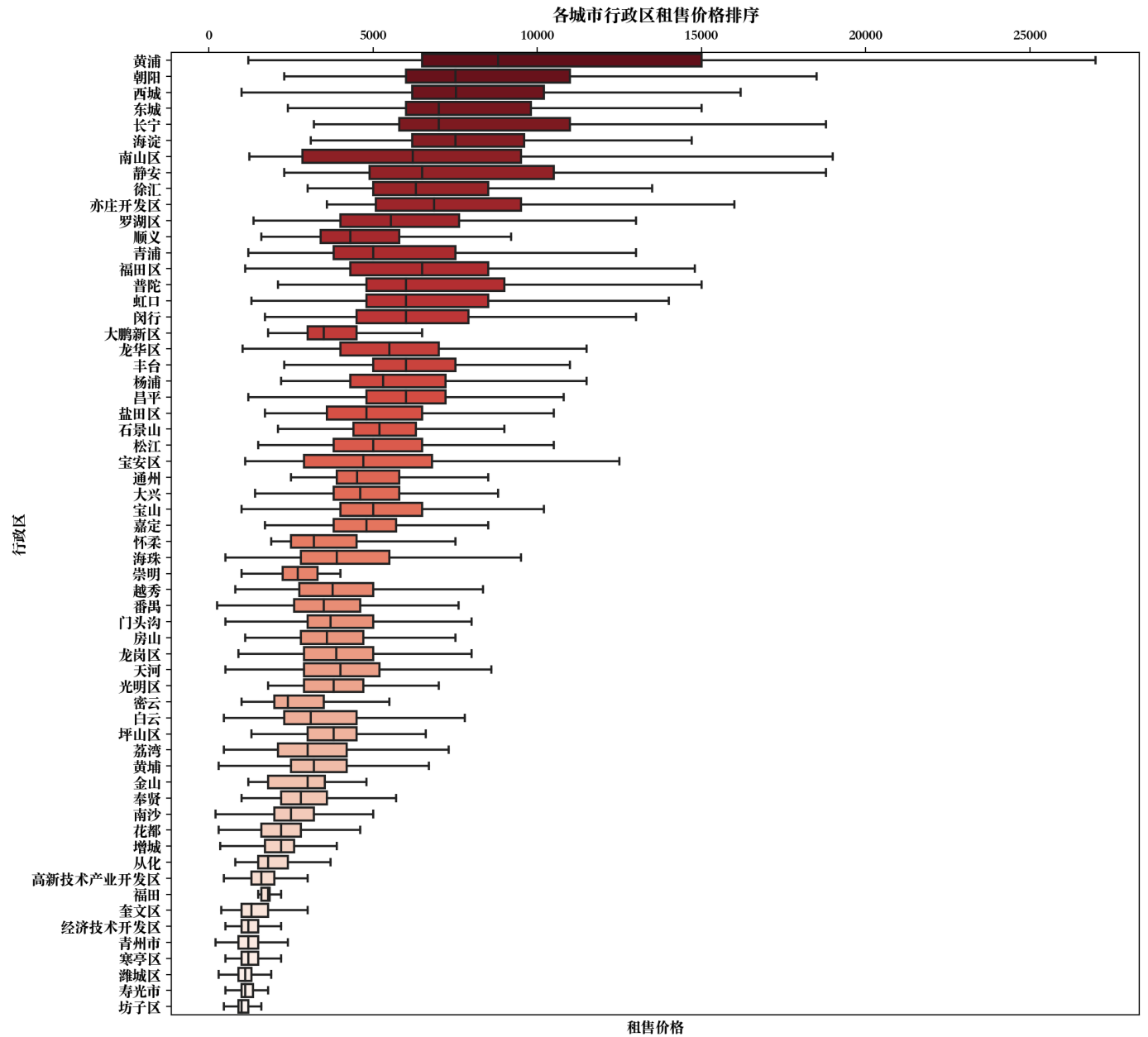
```
region_df = pd.DataFrame(df, columns=["city", "region", "price"])
region_df = df.iloc[
    (-df.groupby("region")["price"].transform("mean")).argsort()
]

mean = region_df["price"].mean()
std = region_df["price"].std()
upper_border = mean + 3 * std

f, ax = plt.subplots(figsize=(12, 12), dpi=200)
g = sns.boxplot(
    y="region",
    x="price",
    data=region_df[region_df["price"] < upper_border],
    ax=ax,
    palette='Reds_r',
    showfliers=False,
)
g.xaxis.tick_top()
g.set_xlabel("租售价格")
g.set_ylabel("行政区")
g.set_title("各城市行政区租售价格排序")
```



```
plt.show()
```



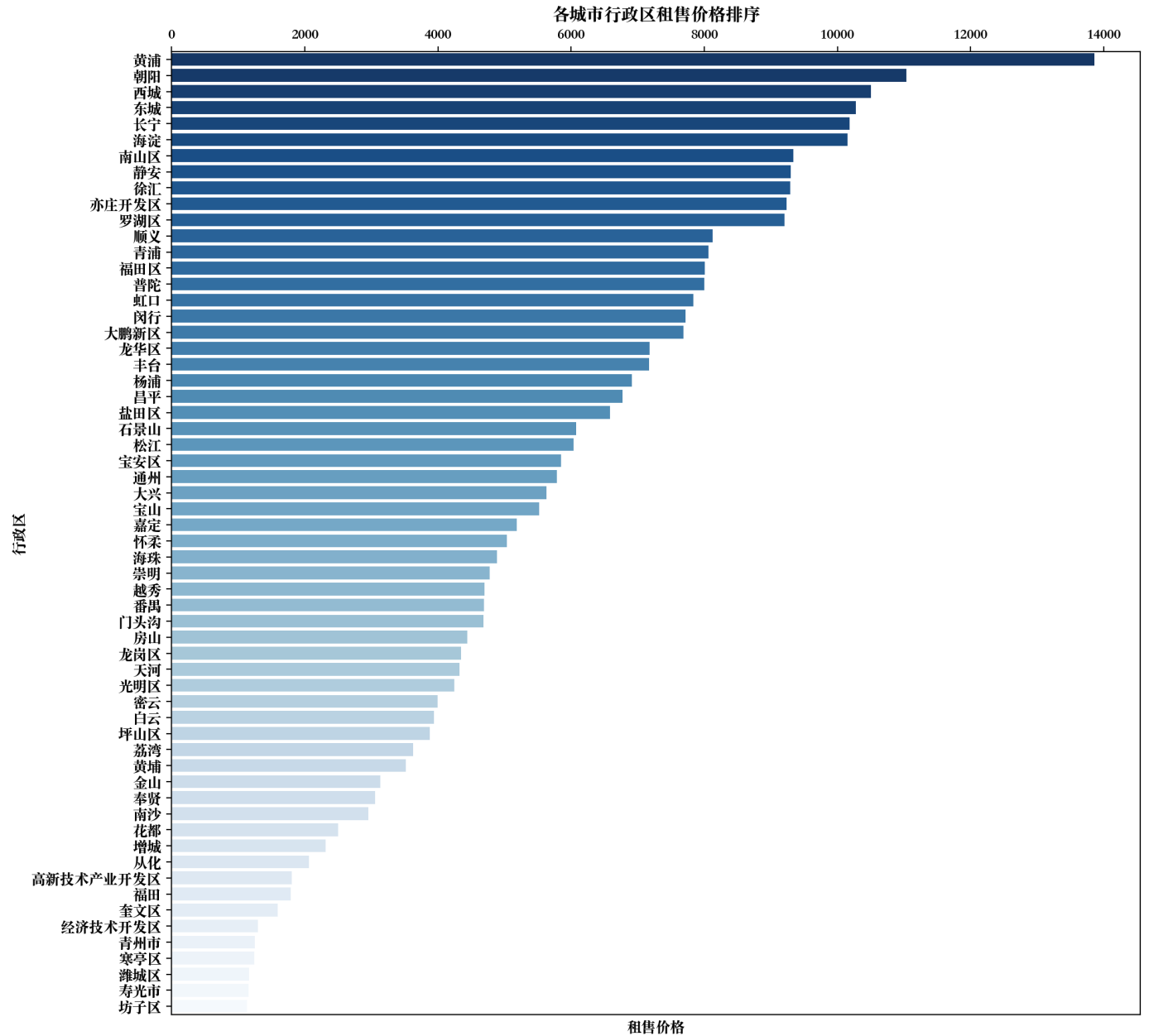
下面绘制均值排布的柱状图.

```
mean_region_df = pd.DataFrame(  
    region_df.groupby("region", as_index=False)["price"]  
    .mean()  
    .sort_values(by='price',ascending=False)  
    .reset_index()  
)  
display(mean_region_df.head())  
  
f, ax = plt.subplots(figsize=(12, 12), dpi=200)  
g = sns.barplot(mean_region_df, y="region", x="price", palette="Blues_r", ax=ax)  
g.xaxis.tick_top()  
g.set_xlabel("租售价格")  
g.set_ylabel("行政区")  
g.set_title("各城市行政区租售价格排序")  
plt.show()
```

```
.dataframe tbody tr th {  
    vertical-align: top;  
}
```

```
.dataframe thead th {
  text-align: right;
}
```

	index	region	price
0	57	黄浦	13856.669130
1	27	朝阳	11036.890411
2	44	西城	10500.345990
3	0	东城	10277.008174
4	48	长宁	10184.131841



上海的黄浦的平均租价明显高于其他行政区划; 排在前列的均是北京上海的行政区, 其次是 深圳的行政区. 排在最后的是潍坊的行政区划, 这与五个城市的经济发展水平相吻合.

朝向与租金分布情况

比较各个城市不同朝向的单位面积租金分布情况, 采用合适的图或表形式进行展示.

比较不同城市租金最高的朝向和租金最低的朝向, 如果不同城市的租金最高和最低朝向不同, 对其进行分析.

首先将朝向字符串拆分成list, 方便后续使用.

```
lighting_df = pd.DataFrame(df, columns=['city', "unit_price", "lighting"])
lighting_df["lighting"] = lighting_df["lighting"].apply(
    lambda x: x.split(" ")
)

exploded_lighting_df = lighting_df.explode('lighting')
exploded_lighting_df=exploded_lighting_df[~exploded_lighting_df['lighting'].isin(['周庄嘉园南里', '海淀北部新区', '新街口西里二区', '万科东第'])]
display(exploded_lighting_df)
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

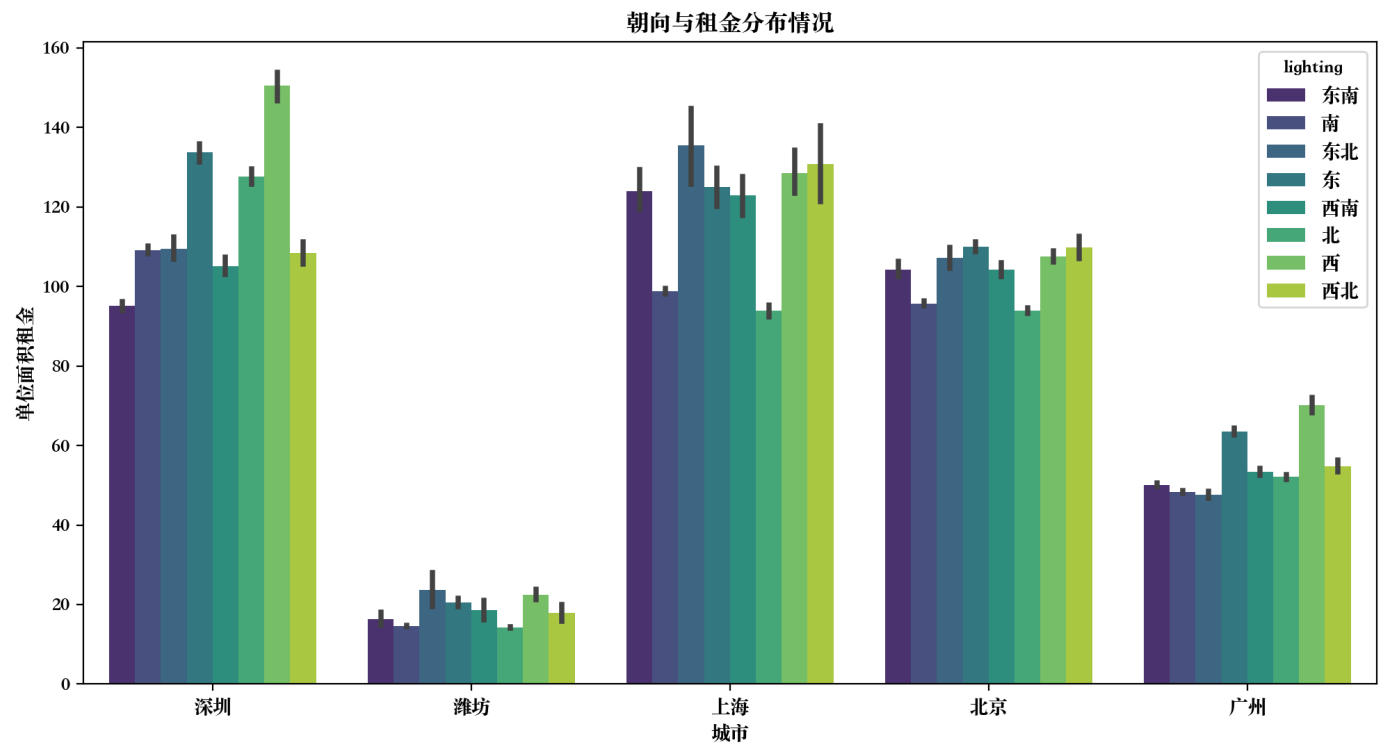
	city	unit_price	lighting
0	深圳	43.48	东南
1	深圳	40.06	南
2	深圳	41.77	南
3	深圳	37.08	南
4	深圳	40.16	东南
...	...	...	...
170034	深圳	111.54	南
170035	深圳	174.32	南
170036	深圳	143.57	东南
170037	深圳	100.00	西南
170038	深圳	288.33	西南

209333 rows × 3 columns

对其根据城市和朝向绘图. 首先绘制柱状图, 分析其绝对数量.

```
fix, ax = plt.subplots(figsize=(12, 6), dpi=200)
g = sns.barplot(
    x="city", y='unit_price', hue="lighting", data=exploded_lighting_df, palette="viridis", ax=ax
)
g.set_xlabel('城市')
g.set_ylabel('单位面积租金')
g.set_title('朝向与租金分布情况')
g.show()
```

```
Text(0.5, 1.0, '朝向与租金分布情况')
```



可以看到, 五个城市的最高单位租金和最低单位租金的朝向不一致.

北京单位面积租金最高的朝向是东和西北, 最低的是北;

上海单位面积租金最高的朝向是东北, 最低的是北;

深圳最高的是西, 最低的是东南;

广州最高的是西, 最低是东北;

潍坊最高是东北, 最低是北.

比较靠北的三个城市北京, 上海和潍坊朝北都是最便宜的, 因为太阳一般从南边照射, 朝北 采光较差, 容易阴冷; 广州朝北的房子也会相对便宜一些, 但是深圳朝北的房子更贵, 因为 深圳在五个城市中的最南边, 且南方靠海, 朝南容易阴湿.

城市人均GDP和单位面积租金的关系

查询各个城市的人均 GDP, 分析并展示其和单位面积租金分布的关系. 相对 而言, 在哪个城市租房的性价比最高?

城市	人均GDP
北京	183937
上海	173757
广州	151162
深圳	174628
潍坊	74686

```
gdp_df = pd.DataFrame(df, columns=["unit_price","city"])

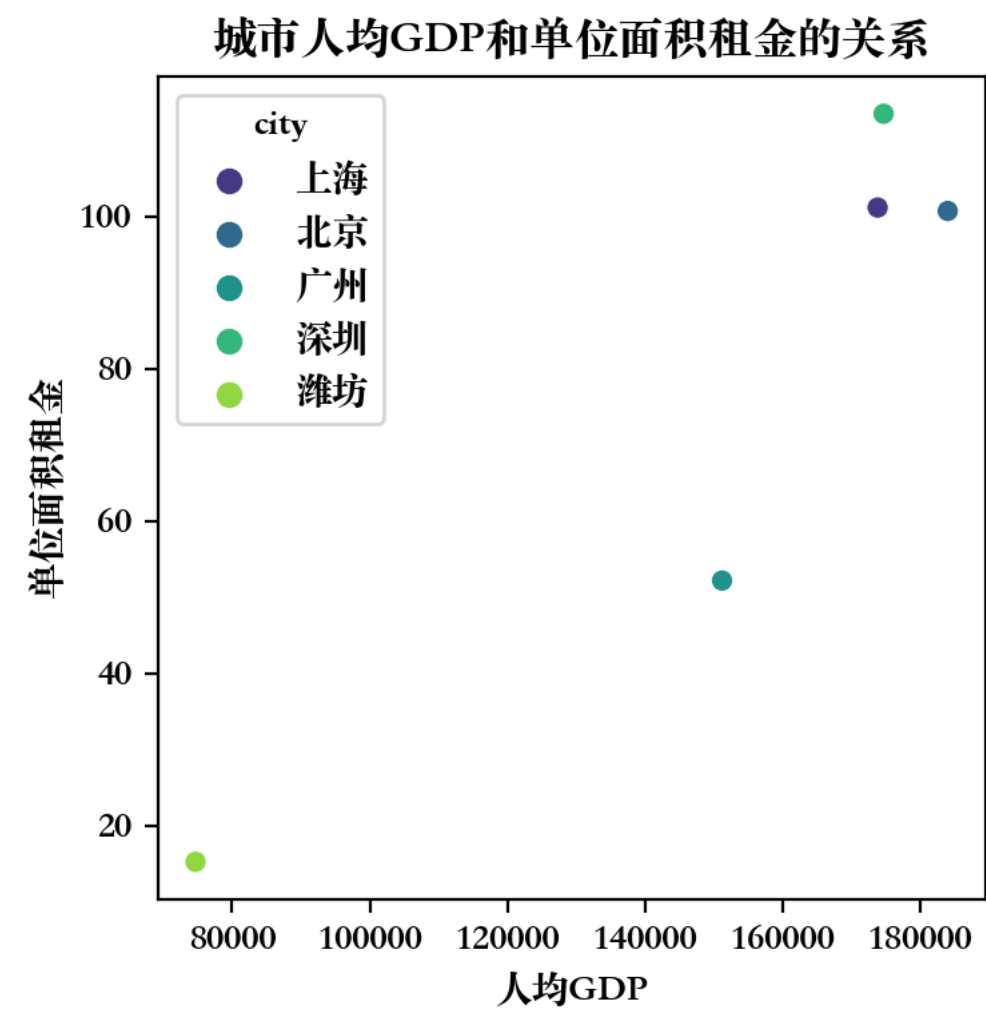
gdp_df = pd.DataFrame(gdp_df.groupby('city')['unit_price'].mean())
gdp = [173757, 183937,151162,174628, 74686]
gdp_df['gdp']=gdp
display(gdp_df)
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	unit_price	gdp
city		
上海	101.186555	173757
北京	100.735278	183937
广州	52.230787	151162
深圳	113.499232	174628
潍坊	15.308537	74686

```
fx, ax = plt.subplots(figsize=(4, 4), dpi=200)
g = sns.scatterplot(
    x="gdp", y="unit_price", hue="city", data=gdp_df, palette="viridis", ax=ax
)
g.set_xlabel('人均GDP')
g.set_ylabel('单位面积租金')
g.set_title('城市人均GDP和单位面积租金的关系')
plt.show()
```



在相同人均GDP的情况下, 单位面积租金越低, 性价比越高. 即在图上与原点连线的斜率越 低, 则性价比越高. 因此相对而言广州的性价比最高.

平均工资和单位面积租金分布的关系

查询各个城市的平均工资, 分析并展示其和单位面积租金分布的关系。相对而言, 哪个城市 的租房负担最重?

首先查询五个城市的2021年平均工资.

城市	月人均工资
北京	17570
上海	12480

城市	月人均工资
广州	8630
深圳	10900
潍坊	6740

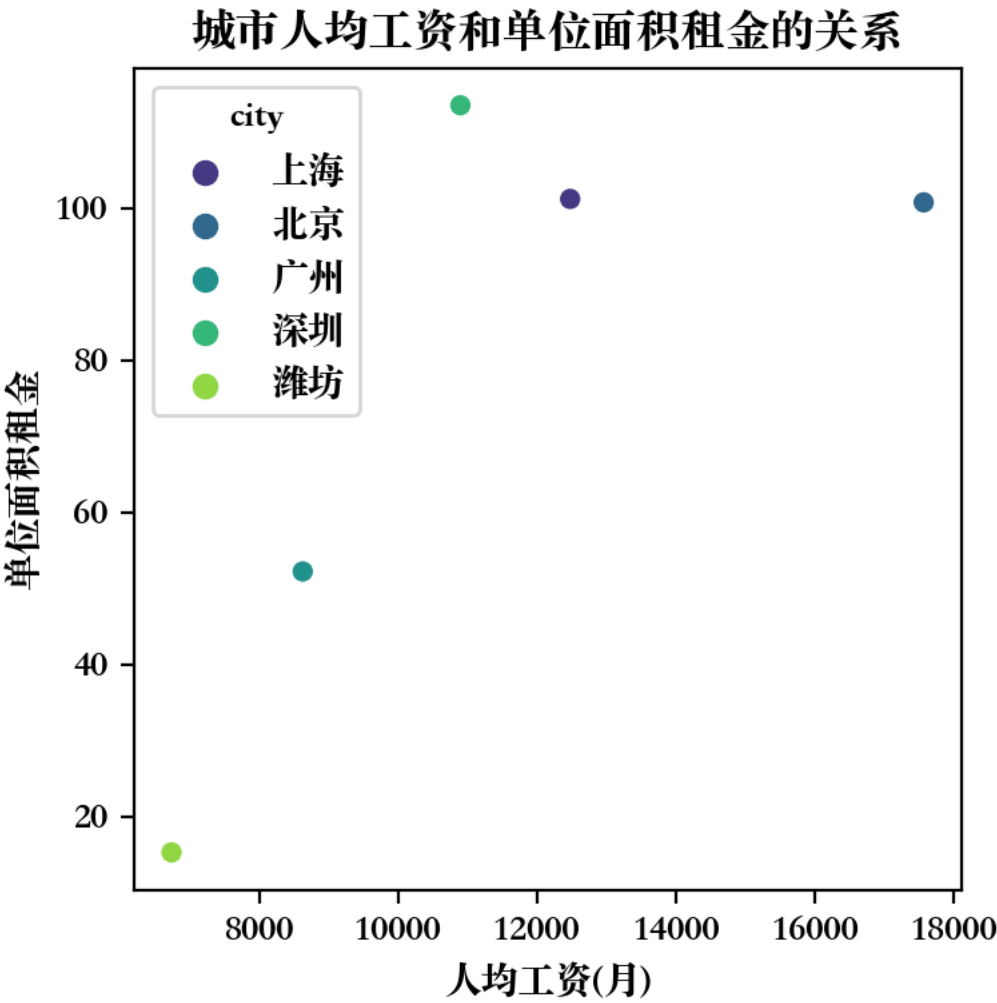
```
salary_df = pd.DataFrame(df, columns=['unit_price','city'])
salary_df = pd.DataFrame(salary_df.groupby('city')['unit_price'].mean())
salary=[12480,17570,8630,10900,6740]
salary_df['salary']=salary
display(salary_df)
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

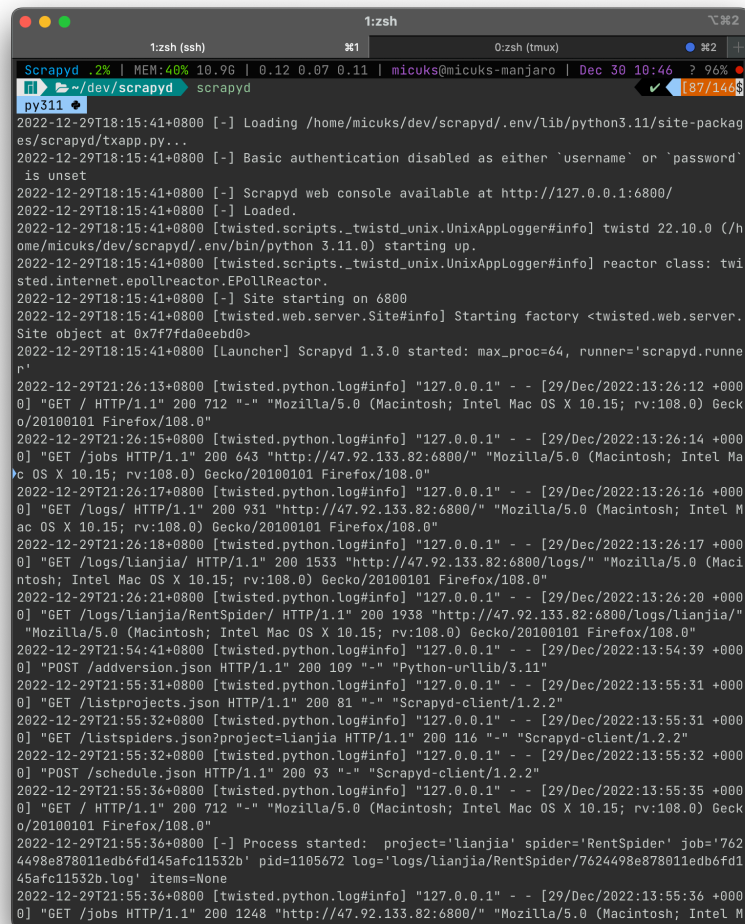
	unit_price	salary
city		
上海	101.186555	12480
北京	100.735278	17570
广州	52.230787	8630
深圳	113.499232	10900
潍坊	15.308537	6740

```
fx, ax = plt.subplots(figsize=(4, 4), dpi=200)
g = sns.scatterplot(
    x="salary",
    y="unit_price",
    hue="city",
    data=salary_df,
    palette="viridis",
    ax=ax,
)
g.set_xlabel("人均工资(月)")
g.set_ylabel("单位面积租金")
g.set_title("城市人均工资和单位面积租金的关系")
plt.show()
```



可以看到人均工资和与单位面积租金的关系和上面人均GDP与单位面积租金的分布类似.

人均工资相同的情况下, 单位面积租金越低, 则房租负担越轻. 因此在五个城市中, 北京的 房租负担最轻.



```
Scrapy 1.3.0 | MEM:40% 10.96 | 0.12 0.07 0.11 | micuks@micuks-manjaro | Dec 30 10:46 ? 96% [87/146$]
~/dev/scrapydev scrapyd
py311
2022-12-29T18:15:41+0800 [-] Loading /home/micuks/dev/scrapydev/.env/lib/python3.11/site-packag
es/scrapydev/txapp.py...
2022-12-29T18:15:41+0800 [-] Basic authentication disabled as either 'username' or 'password'
is unset
2022-12-29T18:15:41+0800 [-] Scrapy web console available at http://127.0.0.1:6800/
2022-12-29T18:15:41+0800 [-] Loaded.
2022-12-29T18:15:41+0800 [twisted.scripts._twistd_unix.UnixAppLogger#info] twisted 22.10.0 (/h
ome/micuks/dev/scrapydev/.env/bin/python 3.11.0) starting up.
2022-12-29T18:15:41+0800 [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twi
sted.internet.epollreactor.EPollReactor.
2022-12-29T18:15:41+0800 [-] Site starting on 6800
2022-12-29T18:15:41+0800 [twisted.web.server.Site#info] Starting factory <twisted.web.server.
Site object at 0x7f7fda0e8bd0>
2022-12-29T18:15:41+0800 [Launcher] Scrapy 1.3.0 started: max_proc=64, runner='scrapydev.runne
r'
2022-12-29T21:26:13+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:12 +000
0] "GET / HTTP/1.1" 200 712 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Geck
o/20100101 Firefox/108.0"
2022-12-29T21:26:15+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:14 +000
0] "GET /jobs HTTP/1.1" 200 643 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel Ma
c OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:17+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:16 +000
0] "GET /logs/ HTTP/1.1" 200 931 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel M
ac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:18+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:17 +000
0] "GET /logs/lianjia/ HTTP/1.1" 200 1533 "http://47.92.133.82:6800/logs/" "Mozilla/5.0 (Maci
ntosh; Intel Mac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:21+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:20 +000
0] "GET /logs/lianjia/RentSpider/ HTTP/1.1" 200 1938 "http://47.92.133.82:6800/logs/lianjia/"
"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:54:41+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:54:39 +000
0] "POST /addversion.json HTTP/1.1" 200 109 "-" "Python-urllib/3.11"
2022-12-29T21:55:31+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:31 +000
0] "GET /listprojects.json HTTP/1.1" 200 81 "-" "Scrapydev-client/1.2.2"
2022-12-29T21:55:32+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:31 +000
0] "GET /listspiders.json?project=lianjia HTTP/1.1" 200 116 "-" "Scrapydev-client/1.2.2"
2022-12-29T21:55:32+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:32 +000
0] "POST /schedule.json HTTP/1.1" 200 93 "-" "Scrapydev-client/1.2.2"
2022-12-29T21:55:36+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:35 +000
0] "GET / HTTP/1.1" 200 712 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Geck
o/20100101 Firefox/108.0"
2022-12-29T21:55:36+0800 [-] Process started: project='lianjia' spider='RentSpider' job='762
4498e878011edb6fd145afc11532b' pid=1105672 log='logs/lianjia/RentSpider/7624498e878011edb6fd1
45afc11532b.log' items=None
2022-12-29T21:55:36+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:36 +000
0] "GET /jobs HTTP/1.1" 200 1248 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel M
```

Figure 1: 运行在 Scrapy 上的爬虫

4 实验总结

通过这次实验,我体验了从零开始设计爬虫,爬取数据到数据分析处理的全部过程,体会到了 python 的方便强大.

在爬虫部署的工具方面,使用了 scrapy 工具,方便的实现了爬虫的服务器端自动化爬取.

但是,由于时间有限,且感染了奥密克戎,深受发烧困扰,没能来得及进行进一步的优化.例如,将使用 jupyter notebook 书写的数据分析部分与前面 latex 书写的爬虫介绍部分更好的组合在一起.也没有实现更多的优化内容,例如 IP 池等,比较遗憾.

总的来说,这次实验在爬虫,数据分析和 Python 之内及之外都学到了许多知识,收获颇丰.

5 附录

代码仓库: [Lianjia](#)